Apache Kafka & Apache Airflow

Concepts • Functions • Roles • Installation • End-to-End Scenario



Lecture note with hands-on installation and a reproducible Docker-based lab

Learning outcomes

By the end of this lecture, students should be able to:

1. Explain Kafka

Describe topics, partitions, brokers, producers, consumers, offsets and consumer groups.

2. Explain Airflow

Describe DAGs, tasks, scheduling, executors, retries, monitoring and dependencies.

3. Distinguish roles

Know why Kafka is an event-stream backbone while Airflow is an orchestration platform.

4. Install locally

Start Kafka and Airflow with Docker / Docker Compose and verify their services.

5. Build a lab

Produce events, consume them, and define an Airflow DAG to control the pipeline.

6. Design a scenario

Map an end-to-end real-time use case to Kafka + processing + storage + Airflow.

Prerequisite: basic Python, Linux commands, Docker, and introductory Spark concepts.

Why streaming data pipelines?

The business value comes from reducing the time between an event and a useful reaction.

Freshness

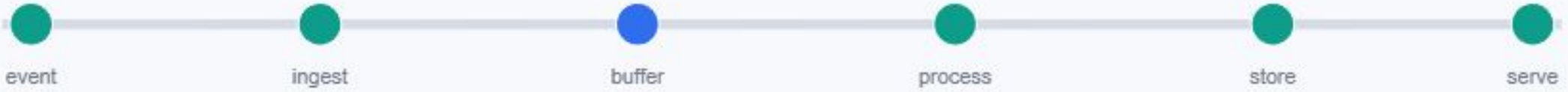
Data arrives at the target as events happen instead of waiting for a large scheduled batch.

Decoupling

Sources and targets do not need to be available at exactly the same moment.

Elastic processing

Producers, processing layers and consumers can scale independently.



Streaming pipeline = source → durable event flow → processing → target, continuously

Source: Confluent streaming-pipeline material provided by the user (Tim Berglund transcript).

Batch vs. streaming

Both are useful. The correct architecture depends on latency, cost and operational complexity.

BATCH

- Collect data for a time window
- Run at intervals: hourly / daily / weekly
- Good for historical ETL and heavy recomputation
- Latency is normally minutes to hours
- Simpler when real-time reaction is unnecessary

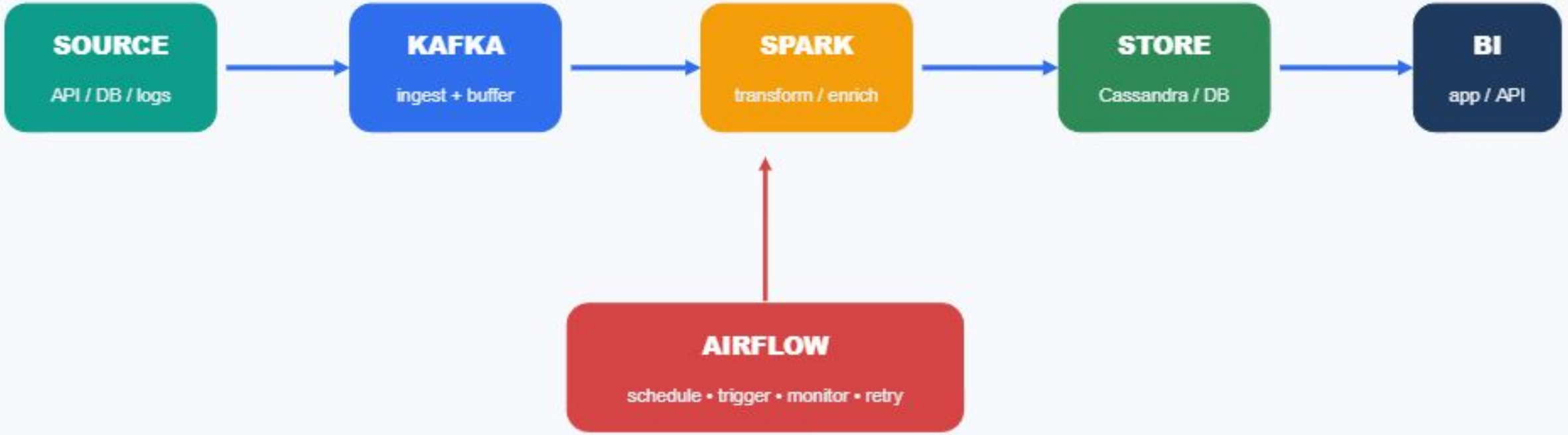
STREAMING

- Process events continuously as they arrive
- Near real-time decisions and alerts
- Good for clickstreams, payments, logs, IoT
- Requires ordering, retries and state handling
- Kafka provides a durable event backbone

Many production systems are hybrid: streaming for freshness + scheduled workflows for control, backfills and maintenance.

Big picture: where each component fits

A practical data platform often separates ingestion, processing, orchestration and storage.



Kafka carries events. Spark processes events. Airflow coordinates jobs and dependencies.

01 Apache Kafka

Distributed event streaming: ingest, store, replay and distribute events reliably at scale.

Kafka in one sentence

Kafka is a distributed event streaming platform for writing, reading, storing and processing event streams.

Write events

Applications publish records through producers.

Store events

Records are durably appended to partition logs for a configurable retention period.

Read / process events

Consumers read independently and can replay older records using offsets.

Important mental model

Kafka is not “just a queue”. It is a distributed, durable log that allows multiple consumers to read the same stored event stream independently.

DISTRIBUTED DURABLE REPLAYABLE SCALABLE FAULT-TOLERANT

Official source: Apache Kafka Quickstart / Getting Started (kafka.apache.org).

Kafka core components

Understand these objects before touching configuration.

Producer

Publishes events to a topic.

Topic

Named stream/category of events.

Partition

Ordered shard of a topic.

Broker

Kafka server storing partitions.

Consumer

Reads records from partitions.

Consumer Group

Consumers cooperate to split work.

Event = key + value + timestamp + headers + metadata (topic / partition / offset)

Topic → partition → offset

Ordering exists within a partition, not globally across all partitions.

Topic: orders

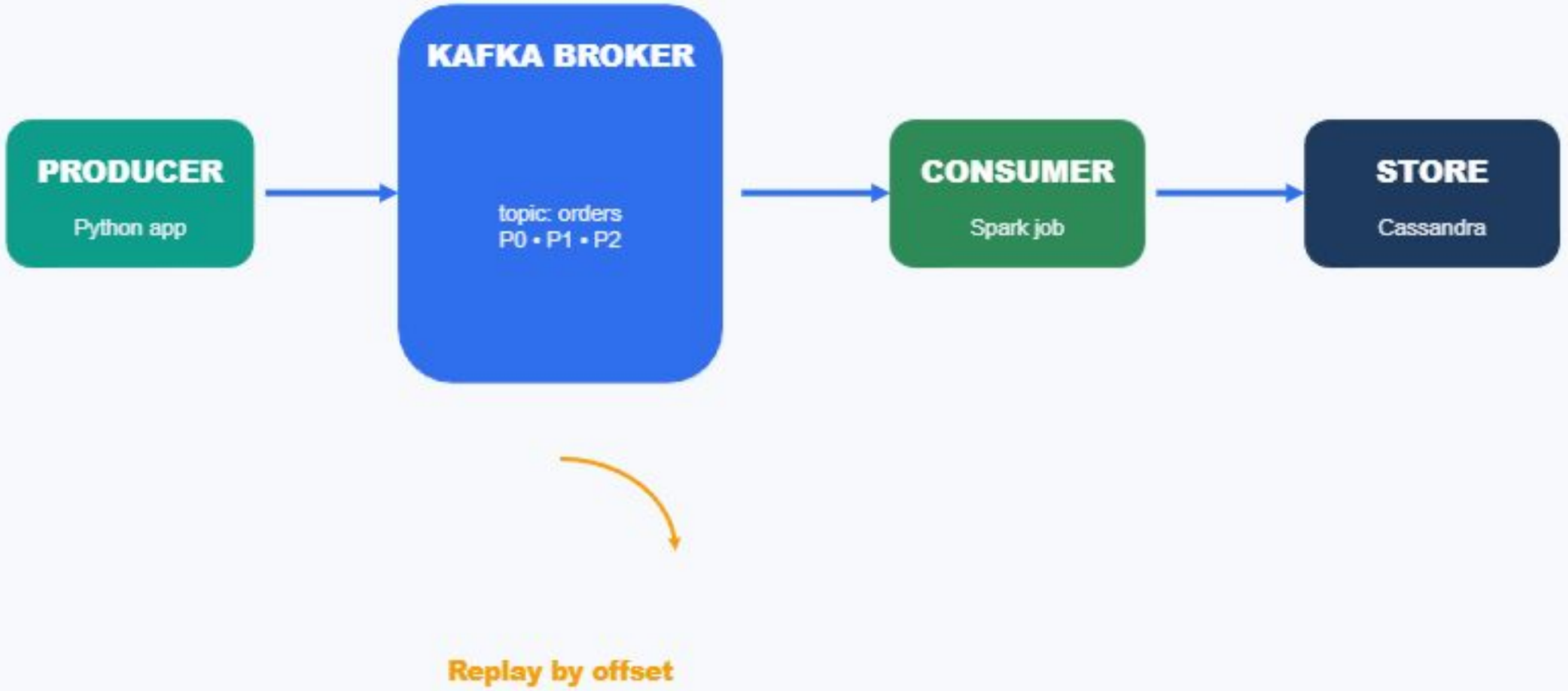
Partition 0	offset 0 order-00	offset 1 order-01	offset 2 order-02	offset 3 order-03	offset 4 order-04	offset 5 order-05
Partition 1	offset 0 order-10	offset 1 order-11	offset 2 order-12	offset 3 order-13	offset 4 order-14	offset 5 order-15
Partition 2	offset 0 order-20	offset 1 order-21	offset 2 order-22	offset 3 order-23	offset 4 order-24	offset 5 order-25

Key design rule: choose a partition key when events for the same entity must preserve relative order (e.g., customer_id or order_id).

Apache Kafka documentation: topic partitions and ordering guarantees.

Producer → broker → consumer flow

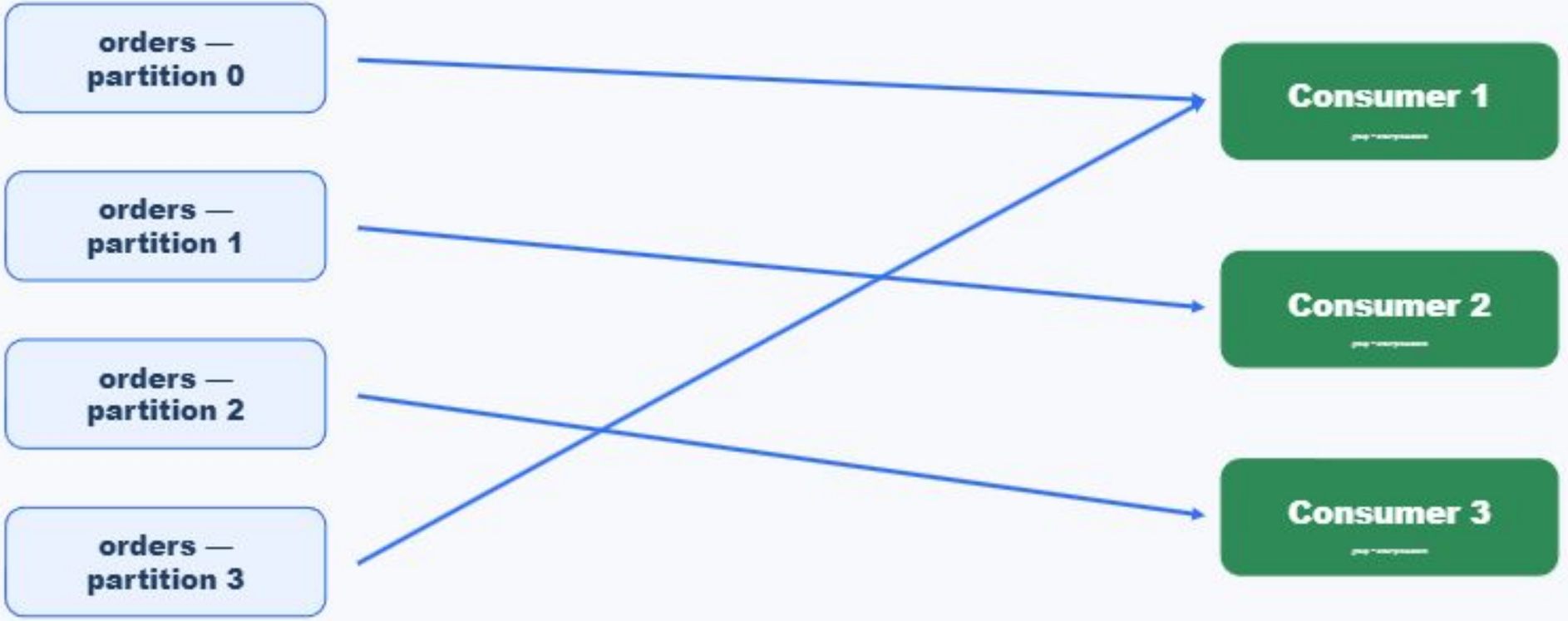
Events move through Kafka while producers and consumers remain loosely coupled.



Kafka persists the event stream, so a consumer can stop and resume without requiring the producer to resend old events.

Consumer groups = horizontal scaling

Within one consumer group, a partition is normally handled by one consumer at a time.



4 partitions + 3 consumers ⇒ one consumer receives two partitions. A 4th consumer can increase parallelism; a 5th would be idle for this topic/group.

Durability, retention and replay

Kafka separates "read progress" from "data deletion".

Retention

Events remain in Kafka according to time/size/compaction policy; consumption does not automatically delete them.

Offsets

A consumer tracks the next position to read for each assigned partition.

Replay

Reset offsets to reprocess older events, rebuild targets, or recover from a processing bug.

Failure example

Spark is offline for 20 minutes → Kafka continues storing incoming events → Spark restarts → resumes from its durable Structured Streaming checkpoint, if records are still retained.

This is one reason Kafka is valuable as a buffer between systems with different availability and throughput.

Kafka: functions and roles in a data platform

Ingestion backbone

Receive high-volume events from many producers.

Buffer / decoupling

Absorb rate differences between source and downstream systems.

Fan-out

Let multiple independent consumers read the same event stream.

Event history

Retain and replay records for recovery or new consumers.

Integration

Connect databases, apps and sinks via Kafka Connect.

Stream processing

Feed Kafka Streams, Spark, Flink or other engines.

When Kafka is — and is not — the right choice

Good fit

- Continuous event streams
- Multiple downstream consumers
- Need replay / durable buffering
- High throughput and horizontal scale
- Event-driven microservices or CDC

Consider simpler alternatives

- Tiny low-volume workflow
- One producer → one simple consumer
- No replay/history requirement
- A normal relational database already solves the problem
- Team cannot operate a distributed streaming platform

Architecture principle: do not use Kafka merely because the system is “big data”; use it when event-stream semantics provide real value.

Kafka installation options

For teaching, Docker is the fastest reproducible path.

Option A — Docker

Official apache/kafka image. Minimal host setup, easy reset, ideal for labs.

Option B — Binary

Download Kafka, install Java 17+, format KRaft storage, then start the broker.

Option C — Managed

Use a managed Kafka service when you want less operational work.

Pinned Kafka lab image: apache/kafka:4.3.1. Verify the image tag before the class.

Kafka 4.x uses KRaft-based metadata management; older ZooKeeper-centric tutorials should not be copied blindly.

Official sources: kafka.apache.org/downloads and kafka.apache.org/quickstart.

Before the lab: start a Docker Engine

Docker CLI alone does not run containers. This Mac has Colima installed.

Terminal

```
# Choose ONE engine: Colima or Docker Desktop
# Colima path (available on this Mac):
colima start --cpu 4 --memory 8
docker context use colima
docker info
docker compose version

# Alternative: install/open Docker Desktop;
# wait for Engine running, then docker info.
# Do not run pull until docker info succeeds.
```

Readiness gate

docker info must show Server details. A CLI version alone does not prove the engine is running.

Airflow resources

Compose v2.14+; Airflow needs ≥4 GB RAM, ideally 8 GB. Full stack needs more.

Windows / Linux

Windows: start Docker Desktop. Linux: start the installed Docker Engine service.

Sources: Docker Desktop for Mac; Airflow 3.3.1 Docker Compose prerequisites.

Troubleshooting: Docker socket missing

Screenshot: docker pull fails before Kafka is downloaded or started.

Terminal

```
# Inspect selected engine and overrides
docker context ls
printenv DOCKER_HOST DOCKER_CONTEXT
colima status

# If using Colima:
colima start --cpu 4 --memory 8
docker context use colima
# If overrides point to a stale engine:
unset DOCKER_HOST DOCKER_CONTEXT
docker info
# Retry docker pull only after Server is ready.
```

What failed?

CLI could not reach unix:///var/run/docker.sock. No broker was started.

On this Mac

Colima was stopped; default context selected the missing socket. Start the intended engine.

Check again

If Docker Desktop is intended, start it and select its context. Do not change socket permissions.

Review of supplied screenshot + local read-only Docker/Colima checks, 2026-10-06.

Hands-on: install Kafka with Docker

Minimal single-node environment for the classroom.

Terminal

```
# 0) Engine must respond before pulling
docker info
# 1) Pull and start the local-only broker
docker pull apache/kafka:4.3.1
docker run -d --name kafka-lab \
  -p 127.0.0.1:9092:9092 apache/kafka:4.3.1
# 2) Verify broker readiness
docker logs --tail 30 kafka-lab
docker exec kafka-lab /opt/kafka/bin/kafka-
topics.sh \
  --list --bootstrap-server localhost:9092
```

Expected result

Engine responds; topic-list command succeeds. Host client: localhost:9092.

Why Docker?

Docker CLI needs a running engine. On this Mac, start Colima first (see previous slide).

Production note

Single node, replication factor 1: no broker failover. Container data needs a volume for persistence.

Command pattern from the official Apache Kafka Docker quickstart.

Hands-on: create a topic, produce and consume

The CLI makes Kafka concepts visible before introducing Python.

Kafka CLI

```
# Create topic
docker exec -it kafka-lab /opt/kafka/bin/kafka-topics.sh \
  --create --topic orders --bootstrap-server localhost:9092 \
  --partitions 3 --replication-factor 1

# Produce events
docker exec -it kafka-lab /opt/kafka/bin/kafka-console-producer.sh \
  --topic orders --bootstrap-server localhost:9092
{"order_id":101,"amount":25.5}
{"order_id":102,"amount":79.0}

# Consume from beginning
docker exec -it kafka-lab /opt/kafka/bin/kafka-console-consumer.sh \
  --topic orders --from-beginning --bootstrap-server localhost:9092
```

Teaching check: ask students to explain what changes if --partitions is 1 vs 3.

Python producer example

Publish JSON events to Kafka.

producer.py

```
# Host Python: install kafka-python in a venv
import json, time, random
from kafka import KafkaProducer

producer = KafkaProducer(
    bootstrap_servers="localhost:9092",
    value_serializer=lambda v: json.dumps(v).encode("utf-8")
)

for order_id in range(1001, 1021):
    event = {
        "order_id": order_id,
        "customer_id": random.randint(1, 5),
        "amount": round(random.uniform(10, 250), 2)
    }
    producer.send("orders", key=str(event["customer_id"]).encode(),
                  value=event).get(timeout=10)
    print("sent", event)
    time.sleep(1)

producer.flush()
producer.close()
```

Key

customer_id is used as the record key to keep a customer's events aligned to a partition.

Value serializer

Converts the Python dictionary into UTF-8 JSON bytes before sending.

flush()

Waits for buffered sends to complete before program exit.

Python consumer example

Read, deserialize and process events as part of a consumer group.

consumer.py

```
import json
from kafka import KafkaConsumer

consumer = KafkaConsumer(
    "orders",
    bootstrap_servers="localhost:9092",
    group_id="order-monitor",
    auto_offset_reset="earliest",
    enable_auto_commit=False,
    value_deserializer=lambda b:
json.loads(b.decode("utf-8"))
)

for msg in consumer:
    order = msg.value
    print(
        f"partition={msg.partition} offset={msg.offset} "
        f"order={order['order_id']}
amount={order['amount']}"
    )
    consumer.commit()
```

group_id

Stores/coordinates consumption progress for a logical application.

auto_offset_reset

Used only when the committed offset is missing or no longer valid; does not rewind each run.

msg.offset

A concrete position inside the partition log.

Mini-lab 1 — Kafka

20–30 minutes

Student tasks

- Start the Kafka Docker container.
- Create topic orders with 3 partitions.
- Run the Python producer for at least 20 records.
- Run two consumer processes with the same group_id.
- Observe partition assignment and offsets.
- Stop one consumer, continue producing, then restart it and discuss what happens.

Evidence to submit

Terminal screenshots + a short explanation of partition, offset and consumer group.

Question

Why can Kafka recover when a consumer is temporarily offline?

Extension

Change the record key and observe how partition distribution changes.

02 Apache Airflow

Workflow orchestration: define DAGs, schedule tasks, handle dependencies, retries and monitoring.

Airflow in one sentence

Apache Airflow is an open-source platform for developing, scheduling and monitoring workflows.

Develop

Define workflows as Python code, including tasks and dependencies.

Schedule / trigger

Run workflows by time, manual trigger, or supported event/asset conditions.

Monitor

Inspect DAG runs, task state, logs, retries and failures in the UI.

Core mental model

Airflow manages the workflow around data jobs. It is not the system that stores Kafka events or performs Spark's continuous stream processing.

Official source: Apache Airflow stable documentation (airflow.apache.org/docs/apache-airflow/stable).

Airflow core concepts: DAG, task, operator, dependency



DAG

The workflow graph: schedule + tasks + dependencies + operational settings.

Task

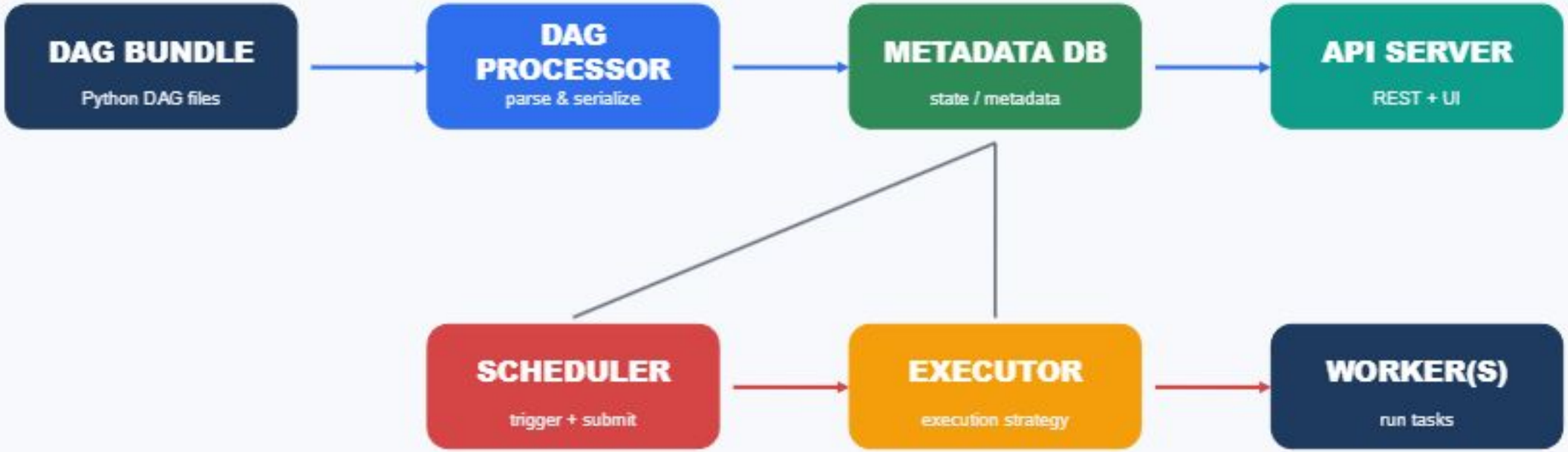
A discrete unit of work that runs as a task instance during a DAG run.

Operator / @task

Reusable templates for defining tasks, including Python, Bash, SQL and provider-specific actions.

Airflow architecture (Airflow 3.x)

A scalable Airflow deployment separates parsing, scheduling, API/UI and task execution.



The executor is configured by the scheduler; production deployments may execute tasks on remote workers.

Official source: Apache Airflow Architecture Overview, stable docs (3.x).

Scheduling, dependencies, retries and observability

Schedule

Cron, timedelta, manual trigger, or supported data/event-aware patterns.

Dependencies

A task normally waits for upstream tasks to satisfy the trigger rule.

Retries

Tasks can retry after failures with configurable retry count and delay.

Logs

Each task instance has execution logs for debugging.

Backfill / rerun

Reprocess historical intervals or rerun specific failed tasks.

UI

Graph/Grid views expose workflow state and dependencies.

Airflow: functions and roles in a data platform

Orchestration

Coordinate multi-step ETL/ELT and ML workflows.

Scheduling

Run data jobs at the correct time or condition.

Dependency management

Enforce task order and branching logic.

Operational recovery

Retry, rerun and backfill failed work.

Integration hub

Invoke Spark, SQL, Python, cloud services and APIs.

Monitoring

Centralize DAG state, task logs and execution history.

Important: Airflow is not Kafka, and it is not Spark Streaming

This distinction prevents a common architecture mistake.

Kafka / Spark Streaming

- Long-running event flow
- Messages/events arrive continuously
- Low-latency stream processing
- Stateful windows, aggregations, joins
- Designed to keep processing as data arrives

Airflow

- Finite tasks inside DAG runs
- Schedules and triggers work
- Tracks dependencies and retries
- Starts/stops/checks external jobs
- Best for workflow control, not record-by-record streaming

Use Airflow to orchestrate a streaming platform — not to replace the streaming platform.

Hands-on: install Airflow with Docker Compose

The official compose quickstart is ideal for a local multi-service lab.

Terminal

```
# Docker Engine ready; Compose v2.14+ required
mkdir -p airflow-lab && cd airflow-lab
curl -LfO \

https://airflow.apache.org/docs/apache-airflow/3.3.1/docker-
compose.yaml
mkdir -p dags logs plugins config
# macOS: .env in this NEW lab directory
printf 'AIRFLOW_UID=50000\n' > .env
# Linux instead: printf 'AIRFLOW_UID=%s\n' "$(id -u)" > .env
docker compose up airflow-init
# Continue only if init exits with code 0
docker compose up -d
docker compose ps
# UI: http://localhost:8080
```

UI

Pinned official quickstart: http://localhost:8080; local demo login airflow / airflow.

Verify

Init exits 0; services are healthy. Reserve ≥4 GB RAM for Airflow, ideally 8 GB.

Teaching note

Separate lab folder. Integration later needs shared networking and Kafka advertised listeners.

Official source: Apache Airflow Docker Compose quickstart / pipeline tutorial.

Your first Airflow DAG

A small TaskFlow example that is easy to inspect in the UI.

hello_pipeline.py

```python
# dags/hello_pipeline.py
import pendulum
from airflow.sdk import dag, task

@dag(
    dag_id="hello_pipeline",
    schedule="0 8 * * *",
    start_date=pendulum.datetime(2026, 1, 1, tz="Asia/Ho_Chi_Minh"),
    catchup=False,
    tags=["lecture"],
)
def hello_pipeline():

    @task
    def extract():
        return {"records": 100}

    @task
    def transform(meta):
        return meta["records"] * 2

    @task
    def load(count):
        print(f"loaded {count} rows")

    load(transform(extract()))

hello_pipeline()
```

schedule

Daily at 08:00 Asia/Ho_Chi_Minh. Scheduled run follows the completed data interval.

TaskFlow

Python functions become tasks using @task; return values can be passed as XCom metadata.

catchup=False

Prevents automatic historical runs for all prior schedule intervals in this introductory lab.

Airflow UI: what students should inspect

The UI is part of the learning outcome, not merely a deployment detail.

DAG list

Is the DAG discovered, enabled, and healthy?

Graph / Grid

Can you explain dependencies and task states?

Task logs

Can you identify the exact error message and retry?

Run history

Which DAG runs succeeded or failed?

Connections

How does Airflow store connection metadata for external services?

Manual trigger

Can a workflow be tested independently of the schedule?

Mini-lab 2 — Airflow

30–40 minutes

- Start Airflow with Docker Compose.
- Create hello_pipeline.py under dags/.
- Confirm the DAG appears in the UI.
- Trigger the DAG manually.
- Inspect task logs and dependency graph.
- Intentionally raise an exception in transform(), observe failure, then configure retry and rerun.

Evidence to submit

DAG code + Graph/Grid screenshot + one task log screenshot.

Question

Why should we avoid putting a forever-running Kafka consumer directly inside a normal Airflow task?

Extension

Add a bounded Python socket check; first connect Airflow and Kafka to a shared Docker network.

03 Kafka + Airflow Together

Use Kafka for continuous event transport; use Airflow to orchestrate, schedule and monitor the jobs around the streaming system.

Scenario: real-time e-commerce order pipeline

Adapted from the referenced end-to-end pipelines: API → Kafka → Spark → Cassandra/PostgreSQL, orchestrated with Airflow.

Business goal

New orders should become queryable for operations and analytics within seconds.

Input

Order events from an application/API: order, customer, amount, product and timestamp.

Output

Validated/enriched order records stored in Cassandra; batch summaries optionally stored in PostgreSQL.

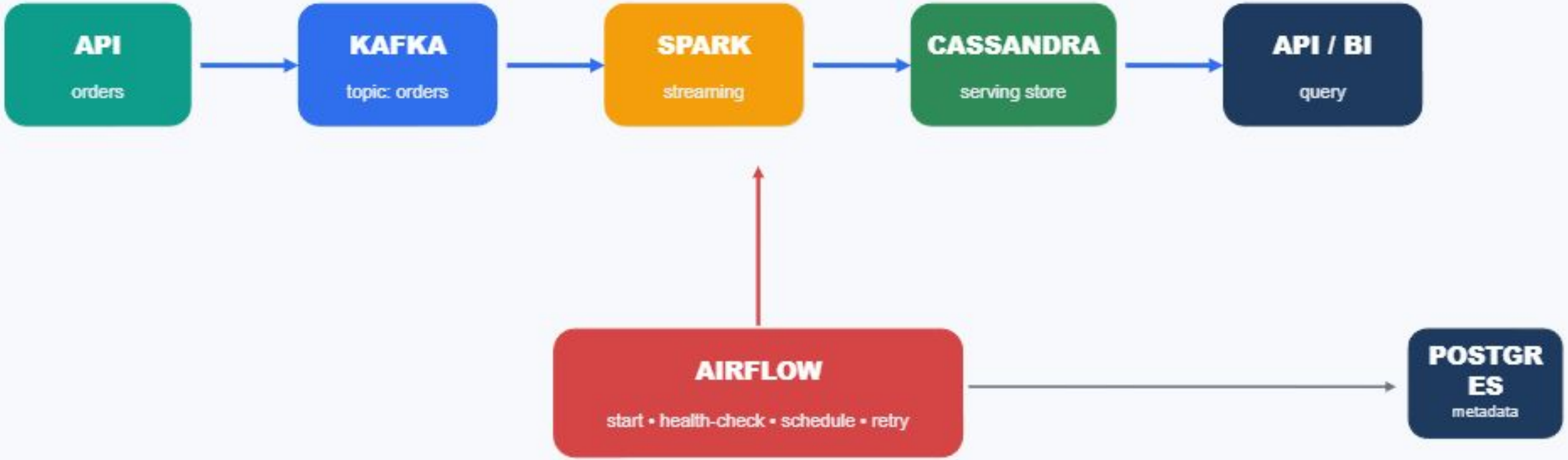
Non-functional requirements

- Continue ingesting if the database is temporarily slow.
- Scale processing independently of the producer.
- Replay events after a bug fix.
- Automatically start/check jobs and retry failed orchestration steps.
- Reproducible local environment for student practice using Docker Compose.

Scenario inspired by github.com/JesusdelCos99/Real-Time-Data-Engineering-Pipeline and Shijun Ju portfolio project.

Scenario architecture

Each technology has one clear responsibility.



Docker Compose provides the common local network and repeatable service startup.

Step-by-step data flow

Follow one order event through the system.

1. **Order API** — Creates JSON event.
2. **Kafka producer** — Publishes to orders topic.
3. **Kafka** — Stores event in a partition.
4. **Spark** — Reads, validates and enriches.
5. **Cassandra** — Stores query-ready record.
6. **Airflow** — Controls lifecycle checks / scheduled jobs.

Airflow does not touch every event; it supervises the surrounding workflow and jobs.

Kafka design for the scenario

Keep the first lab intentionally small, but teach production-oriented decisions.

Topic

orders.raw — original producer events.

Partitions

Start with 3 in the lab. Scale based on expected parallelism and throughput.

Key

customer_id (or order_id) to preserve entity-local ordering.

Retention

Choose a duration that supports recovery and replay requirements.

Schema

Use consistent JSON for the lab; production may use Avro/Protobuf + schema governance.

DLQ / invalid events

Route validation failures to a separate topic such as orders.invalid.

Spark Structured Streaming: consume Kafka and transform

Kafka transports the stream; Spark executes the processing logic.

spark_stream.py

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_json
spark = SparkSession.builder.appName("orders").getOrCreate()
raw = (spark.readStream.format("kafka")
    .option("kafka.bootstrap.servers", "kafka:19092")
    .option("subscribe", "orders.raw")
    .option("startingOffsets", "earliest").load())
schema = "order_id LONG, customer_id LONG, amount DOUBLE"
orders = (raw.select(from_json(col("value").cast("string"),
                               schema).alias("o"))
    .select("o.*").filter(col("amount") > 0))
query = (orders.writeStream.format("console")
    .outputMode("append")
    .option("checkpointLocation", "/checkpoints/orders")
    .start())
query.awaitTermination()
```

Checkpoint

Mount /checkpoints on durable storage. Recovery uses checkpoint progress, not group commits.

Bootstrap server

Shared-network clients: kafka:19092. Advertise this internal listener; host uses localhost:9092.

Sink

Console is diagnostic, not a persistent sink. Add the matching Spark Kafka connector package.

Airflow DAG around the streaming pipeline

A practical DAG can manage bounded control tasks instead of processing every event.



Scheduled branch	Daily: aggregate yesterday's orders → write summary table → data-quality checks → report.
Operational branch	Every 5 minutes: check service health / lag metric → alert if the pipeline is unhealthy.

This pattern uses Airflow for control-plane work while Kafka/Spark stay on the data plane.

Airflow DAG example for the lab

Simplified orchestration code — intentionally finite tasks.

dags/orders_pipeline_control.py

```
from datetime import timedelta
import socket, pendulum
from airflow.sdk import dag, task
@dag(dag_id="orders_pipeline_control",
     schedule="*/10 * * * *",
     start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
     catchup=False, max_active_runs=1,
     default_args={"retries": 2,
                   "retry_delay": timedelta(minutes=1)})
def orders_pipeline_control():
    @task(execution_timeout=timedelta(seconds=30))
    def tcp_check(host, port):
        with socket.create_connection((host, port), timeout=5):
            print(f"TCP reachable: {host}:{port}")
    @task
    def report_ok():
        print("TCP endpoints reachable; verify job and lag separately")
    kafka = tcp_check.override(task_id="check_kafka")("kafka", 19092)
    spark = tcp_check.override(task_id="check_spark")("spark-master", 7077)
    [kafka, spark] >> report_ok()
orders_pipeline_control()
```

Finite tasks

Socket timeout + task timeout bound execution. TCP reachability alone is a basic check.

Service names

Workers must share the Kafka/Spark network. Internal ports differ from published host ports.

Next step

Check Kafka metadata, job state and lag next.

Docker Compose service map for the full lab

Integration checklist — schematic service map, not a ready-to-run Compose file.

Service	Image	Port	Role
kafka	apache/kafka:4.3.1	9092 / 19092	event broker
spark-master	Spark image	7077; 8081*	processing coordinator
spark-worker	Spark image	—	processing worker
cassandra	cassandra:4.1	9042	serving database
postgres	postgres:16	5432	Airflow metadata DB; analytics DB separate
airflow-*	apache/airflow:3.3.1	8080	API/UI, scheduler, workers + Redis

*** Publish Spark UI as 8081:8080; Airflow keeps 8080. Add Redis + DAG processor + triggerer; pin image tags.**

End-to-end classroom demo

Integration exercise: instructor must supply and test the full Compose stack before class.

1. **Start services** — docker compose up -d
2. **Create Kafka topic** — orders.raw with 3 partitions
3. **Start Spark consumer** — readStream from Kafka
4. **Run producer** — topic = orders.raw
5. **Inspect output** — Spark console or Cassandra rows
6. **Open Airflow** — trigger health-check DAG
7. **Failure drill** — stop Spark / restart / observe recovery
8. **Discussion** — where are offsets and checkpoints used?

Failure scenarios to discuss

The best way to understand a data pipeline is to break it safely.

Stop Spark

Kafka continues receiving events. When Spark returns, it resumes from checkpoint/offset progress.

Stop Cassandra

Spark sink may retry/fail depending on connector settings; Kafka retains upstream events.

Stop Kafka

Producers/consumers cannot exchange events; Airflow health check should fail and retry/alert.

Bad event schema

Validation should route/drop according to policy; ideally preserve invalid events for diagnosis.

Duplicate processing

Teach idempotency and delivery semantics; do not assume “exactly once” everywhere automatically.

Airflow down

The continuous Kafka/Spark data path can keep running; orchestration and scheduled controls pause.

Practical assignment

Build a small but explainable end-to-end pipeline.

Required implementation

- Kafka topic with ≥ 3 partitions.
- Python producer generating JSON events.
- Spark Structured Streaming consumer with at least one transformation.
- A persistent sink (Cassandra/PostgreSQL or another approved database).
- Airflow DAG with ≥ 4 finite tasks, dependencies, retry and one validation step.
- Docker Compose plus a README showing how to start and test the system.

Demo

Show events moving through the system live.

Failure test

Stop one service and explain expected recovery.

Explanation

Students must justify Kafka vs Airflow roles, not only show code.

Suggested assessment rubric

Emphasize understanding over container count.

Kafka concepts & topic design	**20%**
Producer/consumer correctness	**15%**
Spark streaming transformation	**20%**
Airflow DAG & orchestration logic	**20%**
Persistence + data validation	**10%**
Docker reproducibility & documentation	**10%**
Demo + explanation	**5%**

Optional bonus: dead-letter topic, schema validation, consumer-lag dashboard, or alerting.

Key takeaways

The exam-level distinctions students should remember.

Kafka

Durable distributed event log. Topics are partitioned; consumers track offsets and can replay.

Airflow

Workflow orchestrator. DAGs describe task order, scheduling, retries and operational control.

Together

Kafka moves events continuously; Airflow controls bounded jobs and operational workflows around the pipeline.

Architecture memory aid

Kafka = data plane for event streams | Airflow = control plane for workflows

A strong data engineer knows not only how to run these tools, but why each one exists.

References and recommended reading

Primary documentation is preferred for installation and version-sensitive commands.

1. Apache Kafka — Quickstart: https://kafka.apache.org/quickstart/

2. Apache Kafka — Downloads / 4.3.x docs: https://kafka.apache.org/downloads/ | https://kafka.apache.org/43/

3. Apache Airflow — Stable documentation: https://airflow.apache.org/docs/apache-airflow/stable/

4. Apache Airflow — Architecture Overview: https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/overview.html

5. Apache Airflow — Docker Compose / tutorial: https://airflow.apache.org/docs/apache-airflow/3.3.1/howto/docker-compose/index.html

6. JesusdelCas99 — Real-Time Data Engineering Pipeline: https://github.com/JesusdelCas99/Real-Time-Data-Engineering-Pipeline

7. Shijun Ju — Airflow + Kafka + Spark + Cassandra pipeline: https://shijunju.com/portfolio/building-a-real-time-data-pipeline-with-apache-airflow-kafka-spark-and-cassandra/

8. Stackademic article supplied by user: Kafka + Airflow + Python orchestration example

9. Confluent streaming pipeline transcript supplied by user (Tim Berglund).

Version note: commands and architecture terminology should be rechecked against official docs before teaching future cohorts.

Hands-on inside the AI-Edu IDE

IDE example — verify the deployed classroom environment before using these samples

Python / PySpark IDE

Mẫu Kafka | Mẫu Airflow | Kiểm tra runtime | ▶ Chạy entry file

EXPLORER

- samples
  - kafka_orders.py
  - airflow_orders_dag.py
- main.py

```
producer.send(TOPIC, key=event["customer_id"],
              value=event).get(timeout=10)


for message in consumer:
    print(message.partition, message.offset,
message.value)
```

Runtime probe

```
Python 3: ready
PySpark: 3.5.x
Kafka client: 2.2.3
Airflow: 3.3.2
```

```
$ output
sent {'order_id': 1001, ...}
partition=1 offset=0 value={'order_id': 1001, ...}
partition=2 offset=0 value={'order_id': 1002, ...}
```

Illustrative UI reconstruction; runtime versions and deployment are not verified by this slide review.

Sample 1 — Kafka: produce, confirm, consume

IDE snippet: namespace replacement, serializers and client setup are supplied by the runner

```
from kafka import KafkaProducer, KafkaConsumer

TOPIC = "__STUDENT_TOPIC__"
producer.send(TOPIC, key="C01", value=event).get(timeout=10)

consumer = KafkaConsumer(
    TOPIC, group_id="student-order-monitor",
    auto_offset_reset="earliest",
    enable_auto_commit=False,
    consumer_timeout_ms=5000)

for msg in consumer:
    print(msg.partition, msg.offset, msg.value)
```

What is real in this lab?

1. KafkaAdminClient creates a three-partition topic.
2. Producer waits for broker acknowledgement with get(timeout=10).
3. Consumer reads actual broker metadata: partition and offset.
4. consumer_timeout_ms ends idle iteration; the runner also needs a total wall-clock timeout.
5. The runner replaces the placeholder with a private topic name.

Canonical source: ai-edu/bigdata-runner/examples/kafka_orders.py

Sample 2 — Airflow 3: author and parse a DAG

The IDE validates DAG structure; the external Airflow service owns scheduling and run history

```python
from airflow.sdk import dag, task

@dag(dag_id="student_orders_pipeline",
     schedule="0 8 * * *", catchup=False,
     default_args={"retries": 2})
def orders_pipeline():
    @task
    def extract(): ...
    @task
    def validate(rows): ...
    @task
    def load(rows): ...
    load(validate(extract()))

dag_object = orders_pipeline()
```




```
$ output
DAG: student_orders_pipeline
Tasks: extract >> validate >> load
Parse OK
```

Then verify the external Airflow service

Parse OK only validates DAG structure. Check deployed DAG discovery, scheduler health and an actual task run separately.

Canonical source: ai-edu/bigdata-runner/examples/airflow_orders_dag.py

Instructor checklist — verify the IDE deployment

Deployment-specific examples: require the actual runner repository and tested classroom services

BUILD	`docker compose build bigdata-spark-runner`	Uses official Airflow constraints for Python 3.10.
START	`docker compose up -d bigdata-spark-runner`	Runner health-check must become healthy.
PROBE	`IDE → Kiểm tra runtime`	Record actual client/Airflow versions; compare against the runner lockfile.
KAFKA	`IDE → Mẫu Kafka → Chạy entry file`	Require sent events plus real partition/offset output.
AIRFLOW	`IDE → Mẫu Airflow → Chạy entry file`	Require DAG id and extract >> validate >> load.
SERVICE	`Stage 9 → airflow health`	Require healthy metadatabase and scheduler.

Controlled sources

`Dockerfile + requirements-lab.txt + examples/*.py + docs/kafka-airflow-lab.md`